

PRONINX

The Definitive Handbook

"Initially, we were building this for RISC-V, and then we just died."

1. Introduction & Philosophy

PRONINX is a 64-bit operating system based on the classic Unix v6 design, targeting the x86_64 architecture. It is designed with a monolithic mindset where the kernel and the core userland are inseparable.

The Unified Core Philosophy

In PRONINX, the boundary between the kernel and the base system utilities is intentionally tight. The core userland is not a separate entity that can be swapped out easily; it is built to match the kernel's internal structures and ABI exactly. Attempting to decouple them would require rewriting a significant portion of the kernel's logic. They are two halves of the same whole.

Non-POSIX Compliance

PRONINX is **not** POSIX-compliant. It follows the Unix v6 tradition. It does not aim for compatibility with modern Linux or BSD standards. It is its own beast, with its own syscalls and its own way of doing things.

2. System Architecture

Boot Sequence

The boot process is a rigid three-stage transition:

- **Stage 1 (Real Mode):** Wakes up the CPU in 16-bit mode, handles A20, and jumps to 32-bit.

- **Stage 2 (Protected Mode):** Reads the ELF kernel header from the disk (Sector 32+) and loads segments into physical memory.
- **Stage 3 (Long Mode Transition):** Sets up the initial 4-level paging (PML4), enables Long Mode, and jumps to the 64-bit kernel entry point.

Memory Layout

- **Kernel Space:** Resides at the high-half address `0xffffffff80000000`.
- **User Space:** Starts at virtual address `0x0`.
- **Paging:** 4-level paging is mandatory. The kernel manages physical memory in 4KB chunks.

Process Management

The system supports up to 64 concurrent processes. Scheduling is handled via a Round-Robin algorithm. Process states include `UNUSED`, `EMBRYO`, `SLEEPING`, `RUNNABLE`, `RUNNING`, and `ZOMBIE`.

3. ABI Specification (x86_64)

System Call Interface

System calls are triggered via software interrupt `0x30`.

- **Interrupt:** `int $0x30`
- **Registers:**
 - `%rax`: System Call Number (In) / Return Value (Out)
 - `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, `%r9`: Arguments 1-6

Core System Calls

Number	Name	Description
0	SYS_exit	Terminate the calling process
1	SYS_read	Read data from a file descriptor
2	SYS_write	Write data to a file descriptor

3	SYS_open	Open a file or device
5	SYS_fork	Create a new process
8	SYS_exec	Replace current process with a new program
9	SYS_sbrk	Grow/shrink process memory
18	SYS_getdents	Read directory entries

4. Development Guide

Writing Programs

Programs must be written against the PRONINX core library. Use the provided headers in the system source tree.

```
#include "user.h"

int main(int argc, char *argv[]) {
    printf("Welcome to PRONINX\n");
    exit();
}
```

Note: Every program must end with an explicit `exit()` call to return control to the kernel.

Building the System

1. **Build:** Run `make` to generate the kernel image and the FAT32-based filesystem image.
2. **Run:** Run `make qemu` to boot the system in QEMU.
3. **Configure:** Adjust system limits (like `NPROC`) in the kernel parameter headers.

5. Technical Constants

File Types

- `T_DIR` : 1 (Directory)

- `T_FILE` : 2 (Regular File)
- `T_DEVICE` : 3 (Device Node)

Terminal Control

PRONINX supports basic terminal IOCTLs following the termios structure.

- `ICANON` : 0000002
- `ECHO` : 0000010